

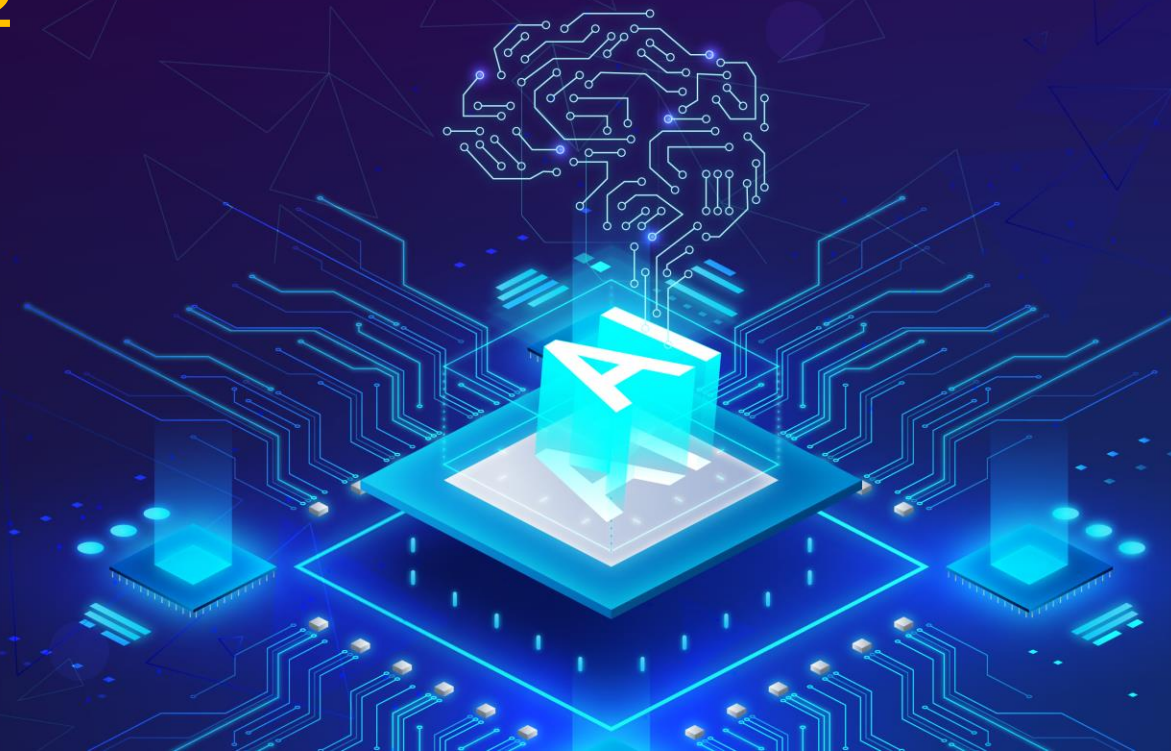


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Nhóm chuyên môn Trí tuệ nhân tạo

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Hướng dẫn triển khai bài tập lớn - Phần 2



Xây dựng chương trình lọc thư rác

MỤC TIÊU



Sau bài học này, người học có thể:

Nắm được cách xây dựng **chương trình**
lọc thư rác dựa trên thuật toán Naïve
Bayes

1. Xây dựng chương trình học thư rác

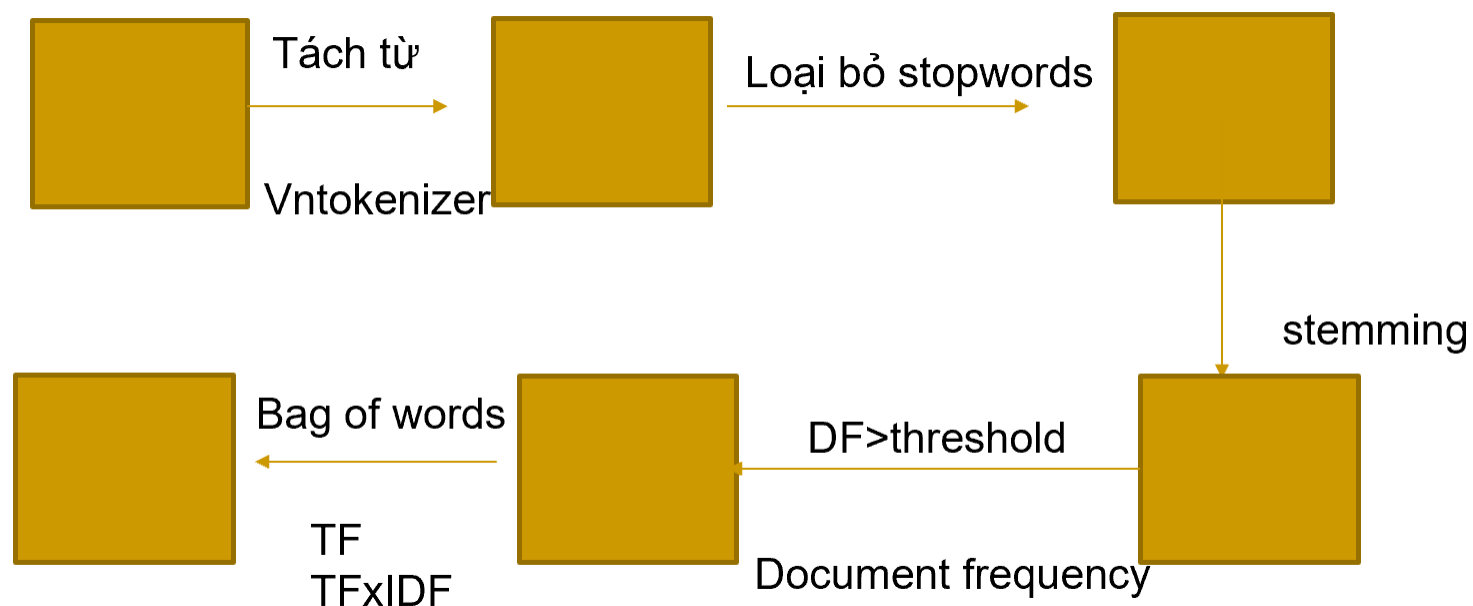
1. XÂY DỰNG CHƯƠNG TRÌNH LỌC THƯ RÁC



- Dữ liệu:
 - [Email Spam Classification Dataset CSV \(kaggle.com\)](#)
 - [Spam Mails Dataset \(kaggle.com\)](#)
- Tiền xử lý:
 - Xây dựng từ điển: Giảm chiều
 - Biểu diễn văn bản theo vector

1. XÂY DỰNG CHƯƠNG TRÌNH LỌC THƯ RÁC

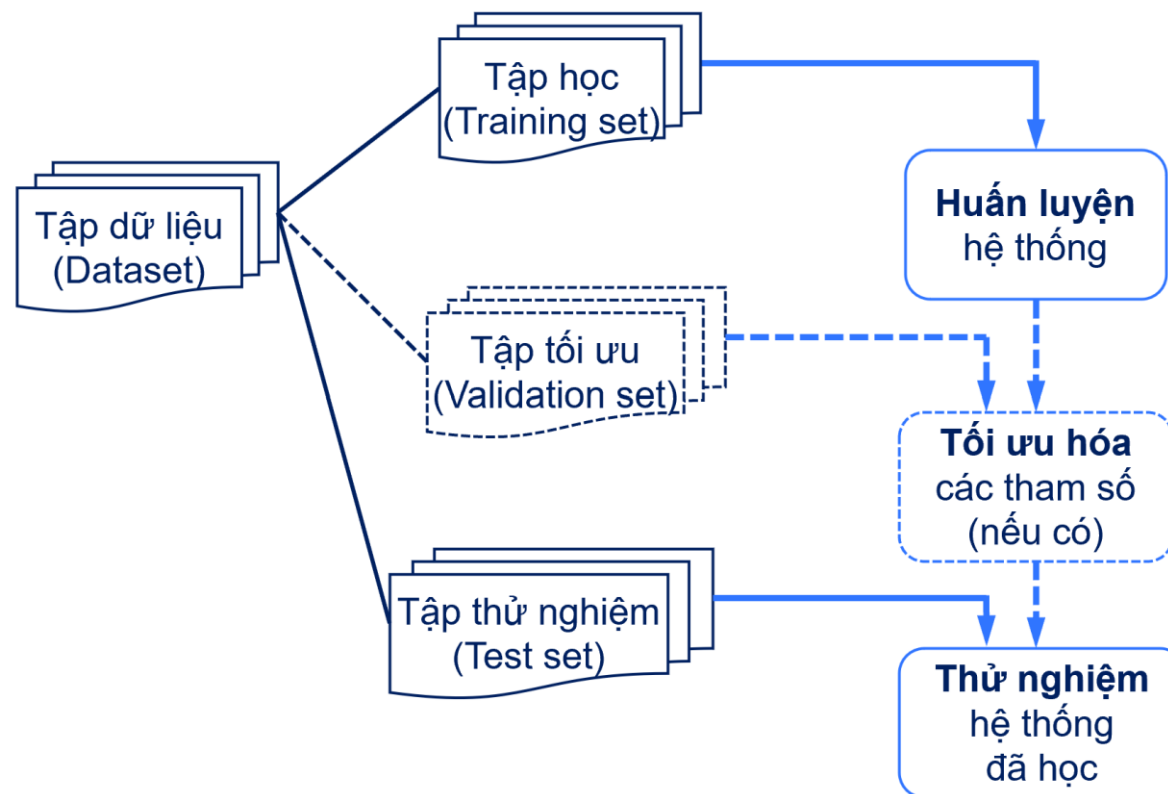
- Quy trình xây dựng từ điển và biểu diễn vector:



Hình 1: Xây dựng từ điển và biểu diễn

1. XÂY DỰNG CHƯƠNG TRÌNH LỘC THƯ RÁC

- Quy trình xây dựng và đánh giá mô hình học máy:



Hình 2: Quy trình xây dựng mô hình học máy

1. XÂY DỰNG CHƯƠNG TRÌNH LỌC THƯ RÁC



- Biểu diễn bài toán phân loại văn bản
 - Tập học D , trong đó mỗi ví dụ học là một biểu diễn văn bản gắn với một nhãn lớp: $D = \{(d_k, c_i)\}$
 - Một tập các nhãn lớp xác định: $C = \{c_i\}$. 2 lớp: spam và non-spam
- Giai đoạn học
 - Từ tập các văn bản trong D , trích ra tập các từ khóa $T = \{t_j\}$
 - Gọi D_{c_i} là tập các văn bản trong D có nhãn lớp c_i
 - Đối với mỗi phân lớp c_i
 - Tính giá trị xác suất trước của phân lớp c_i : $P(c_i) = \frac{|D_{c_i}|}{|D|}$
 - Đối với mỗi từ khóa t_j , tính xác suất từ khóa t_j xuất hiện đối với lớp c_i

$$P(t_j | c_i) = \frac{\left(\sum_{d_k \in D_{c_i}} n(d_k, t_j) \right) + 1}{\left(\sum_{d_k \in D_{c_i}} \sum_{t_m \in T} n(d_k, t_m) \right) + |T|}$$

$(n(d_k, t_j))$: số lần xuất hiện của từ khóa t_j trong văn bản d_k

1. XÂY DỰNG CHƯƠNG TRÌNH LỌC THƯ RÁC



- Sử dụng thư viện Scikit-learn để:
 - Tiền xử lý và biểu diễn vector
 - Chia dữ liệu: Dữ liệu huấn luyện và dữ liệu đánh giá
 - Huấn luyện và dự đoán với thuật toán Naïve Bayes

1. XÂY DỰNG CHƯƠNG TRÌNH LỌC THƯ RÁC

- Import các thư viện từ sklearn

```
import matplotlib.pyplot as plt
import numpy as np

#from sklearn.datasets import load_files
from pyvi import ViTokenizer # Tách từ tiếng Việt

import sklearn.naive_bayes as naive_bayes
from sklearn.datasets import load_files
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.feature_extraction.text import TfidfTransformer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.model_selection import ShuffleSplit
from sklearn.model_selection import learning_curve

%matplotlib inline
```

1. XÂY DỰNG CHƯƠNG TRÌNH LỌC THƯ RÁC



- Tiền xử lý

```
# Transforming data
# Chuyển hoá dữ liệu text về dạng vector tfidf
#     - loại bỏ từ dừng
#     - sinh từ điển
module_count_vector = CountVectorizer(stop_words=stopwords)
model_rf_preprocess = Pipeline([('vect', module_count_vector),
                                ('tfidf', TfidfTransformer()),
                                ])

# Hàm thực hiện chuyển đổi dữ liệu text thành dữ liệu số dạng ma trận
# Input: Dữ liệu 2 chiều dạng numpy.array, mảng nhãn id dạng numpy.array

# Tiền xử lý với Bag of words
data_bow = module_count_vector.fit_transform(data_train.data, data_train.target)
# Tiền xử lý với TF-IDF
data_tfidf = model_rf_preprocess.fit_transform(data_train.data, data_train.
↪target)
```

1. XÂY DỰNG CHƯƠNG TRÌNH LỘC THƯ RÁC



- Chia dữ liệu: Dữ liệu huấn luyện và đánh giá

```
from sklearn.model_selection import train_test_split

# chia dữ liệu thành 2 phần sử dụng hàm train_test_split.
test_size = 0.2
# Bow
X_train_bow, X_test_bow, y_train_bow, y_test_bow = train_test_split(data_bow,
    ↳data_train.target, test_size=test_size, random_state=30)
# Tf-idf
X_train_tfidf, X_test_tfidf, y_train_tfidf, y_test_tfidf =
    ↳train_test_split(data_tfidf, data_train.target, test_size=test_size,
    ↳random_state=30)

# hiển thị một số thông tin về dữ liệu
print("Dữ liệu training = ", X_train_bow.shape, y_train_bow.shape)
print("Dữ liệu testing = ", X_test_bow.shape, y_test_bow.shape)

print()
print("Danh sách nhãn và id tương ứng: ", [(idx, name) for idx, name in
    ↳enumerate(data_train.target_names)] )
```

1. XÂY DỰNG CHƯƠNG TRÌNH LỌC THƯ RÁC



- Huấn luyện và dự đoán:

```
print("- Training ...")

# X_train.shape
print("- Train size = {}".format(X_train_bow.shape))
model_MNB = naive_bayes.MultinomialNB(alpha= 0.1)
model_MNB.fit(X_train_bow, y_train_bow)

print("- model_MNB - train complete")
```

```
print("- Testing ...")
y_pred_bow = model_MNB.predict(X_test_bow)
print("- Acc = {}".format(accuracy_score(y_test_bow, y_pred_bow)))
```

1. Bài học về cách xây dựng **chương trình lọc thư rác dựa trên thuật toán Naïve Bayes.**
2. **Sử dụng thư viện Scikit-learn.**

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

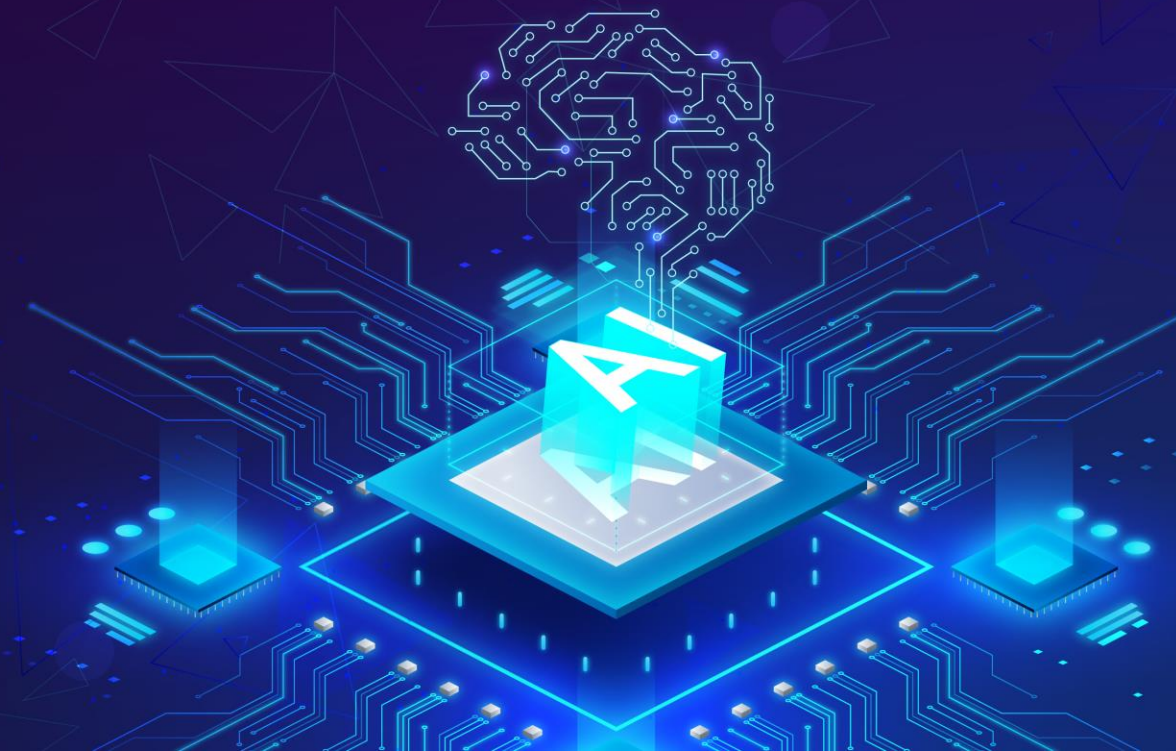
Hướng dẫn triển khai bài tập lớn - Phần 2

Biên soạn:

TS. Ngô Văn Linh

Trình bày:

TS. Ngô Văn Linh



NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Bài học tiếp theo:

Hướng dẫn triển khai bài tập lớn - Phần 3

Tài liệu tham khảo:

- [1] T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
- [2] A. Kontorovich and Weiss. A Bayes consistent 1-NN classifier. Proceedings of the 18th International Conference on Artificial Intelligence and Statistics (AISTATS). JMLR: W&CP volume 38, 2015.
- [3] A. Guyader, N. Hengartner. On the Mutual Nearest Neighbors Estimate in Regression. *Journal of Machine Learning Research* 14 (2013) 2361-2376.
- [4] L. Gottlieb, A. Kontorovich, and P. Nisnevitch. Near-optimal sample compression for nearest neighbors. *Advances in Neural Information Processing Systems*, 2014.